Numerical Solution of a Reaction-Diffusion PDE: Design, Analysis, and Results

Course: *TP TM — Numerical Methods for PDEs*

Date: *April 2026*

Language: *Python 3 / NumPy*

Abstract

We study the numerical solution of the linear reaction-diffusion equation

$$u_t = u_{xx} - \alpha\, u, \quad x \in (0,1),\ t > 0,$$

subject to homogeneous Dirichlet boundary conditions and two distinct initial profiles. Three finite-difference schemes are implemented and compared: the explicit Forward-Time Centred-Space (FTCS) method, the implicit Backward-Time Centred-Space (BTCS) method, and the Crank-Nicolson (CN) scheme. Stability is analysed via von Neumann's method; accuracy is verified against the known exact solution for the first initial condition. Numerical experiments confirm the predicted convergence orders — O(dt + dx²) for FTCS and BTCS, O(dt² + dx²) for CN — and illustrate the qualitative difference between conditionally and unconditionally stable schemes.

1. Problem Formulation

1.1 Governing Equation

The PDE to be solved is

$$u_t(x,t) = u_{xx}(x,t) - \alpha\, u(x,t), \tag{1}$$

where $\alpha \geq 0$ is a reaction (decay) coefficient. This is a *reaction-diffusion* equation: diffusion spreads spatial gradients while the reaction term $-\alpha u$ damps the amplitude uniformly in space.

Boundary conditions (Dirichlet, homogeneous):

$$u(0,t) = 0, \quad u(1,t) = 0. \tag{2}$$

Initial condition:

$$u(x,0) = f(x). \tag{3}$$

Two initial profiles are considered:

Label	Definition
f1	$f(x) = \sin(2\pi x)$
f2	$f(x) = 2x$ if $x \le \frac{1}{2}$, $2(1-x)$ if $x > \frac{1}{2}$

`f1` is a pure sine mode and admits a closed-form solution; `f2` is a piecewise-linear tent function whose Fourier decomposition contains infinitely many odd modes.

1.2 Exact Solution for f1

Since $\sin(2\pi x)$ is an eigenfunction of $-\partial_{xx}$ on $(0,1)$ with eigenvalue $\lambda = 4\pi^2$, substituting $u(x,t) = g(t)\sin(2\pi x)$ into (1) gives

$$g'(t) = -(4\pi^2 + \alpha)\, g(t) \Rightarrow g(t) = e^{-(4\pi^2+\alpha)t}. \tag{4}$$

Hence the exact solution is

$$u(x,t) = e^{-(4\pi^2+\alpha)\,t}\sin(2\pi x). \tag{5}$$

The solution decays exponentially at rate $4\pi^2 + \alpha \approx 39.5 + \alpha$. Larger α accelerates amplitude decay without altering the spatial shape. This is a key insight: the reaction term does **not** couple spatial modes — it only amplifies the natural exponential decay already present from diffusion.

1.3 Behaviour for f2

Fourier sine series and derivation of coefficients. We expand f_2 in the orthonormal basis $\{\sin(n\pi x)\}_{n\ge 1}$ on $(0,1)$:

$$f_2(x) = \sum_{n=1}^{\infty} b_n \sin(n\pi x), \qquad b_n = 2\int_0^1 f_2(x)\sin(n\pi x)\,\mathrm{d}x. \tag{6}$$

Splitting at $x = 1/2$ and using $f_2(x) = 2x$ on $[0, \frac{1}{2}]$ and $2(1-x)$ on $(\frac{1}{2}, 1]$:

$$b_n = 2\left[\int_0^{1/2} 2x\sin(n\pi x)\,dx + \int_{1/2}^{1} 2(1-x)\sin(n\pi x)\,dx\right]. \tag{7}$$

Each integral is evaluated by parts, using $\int x\sin(ax)\,dx = -\frac{x}{a}\cos(ax) + \frac{1}{a^2}\sin(ax)$. For the first piece:

$$\int_0^{1/2} 2x\sin(n\pi x)\,dx = \left[-\frac{2x}{n\pi}\cos(n\pi x) + \frac{2}{n^2\pi^2}\sin(n\pi x)\right]_0^{1/2} \tag{8}$$

$$= -\frac{1}{n\pi}\cos\frac{n\pi}{2} + \frac{2}{n^2\pi^2}\sin\frac{n\pi}{2}.$$

For the second:

$$\int_{1/2}^{1} 2(1-x)\sin(n\pi x)\,dx = \left[-\frac{2(1-x)}{n\pi}\cos(n\pi x) - \frac{2}{n^2\pi^2}\sin(n\pi x)\right]_{1/2}^{1} \tag{9}$$

$$= \frac{1}{n\pi}\cos\frac{n\pi}{2} - \frac{2}{n^2\pi^2}\sin\frac{n\pi}{2} + \frac{2}{n^2\pi^2}\sin\frac{n\pi}{2}\cdot(-1).$$

Summing both pieces and simplifying:

$$b_n = \frac{4}{n^2\pi^2}\left[2\sin\frac{n\pi}{2} - \sin(n\pi)\right] = \frac{8}{n^2\pi^2}\sin\frac{n\pi}{2}. \tag{10}$$

For **even** n: $\sin(n\pi/2) = 0$, so $b_n = 0$.
For **odd** $n = 2k+1$: $\sin\left(\frac{(2k+1)\pi}{2}\right) = (-1)^k$, so

$$b_{2k+1} = \frac{8}{(2k+1)^2\pi^2}(-1)^k, \quad k = 0, 1, 2, \ldots \tag{11}$$

Hence the Fourier sine series is

$$f_2(x) = \frac{8}{\pi^2}\sum_{k=0}^{\infty}\frac{(-1)^k}{(2k+1)^2}\sin\left((2k+1)\pi x\right). \tag{12}$$

Solution for f2. Each mode $\sin(n\pi x)$ evolves independently; applying the decay rate $n^2\pi^2 + \alpha$ gives

$$u(x,t) = \frac{8}{\pi^2}\sum_{k=0}^{\infty}\frac{(-1)^k}{(2k+1)^2}\,e^{-((2k+1)^2\pi^2+\alpha)\,t}\sin\left((2k+1)\pi x\right). \tag{13}$$

In practice the series is truncated at $K \approx 50$ terms, since the amplitude of mode $n = 2k+1$ decays as $(2k+1)^{-2}$ and the exponential suppresses high modes rapidly for $t > 0$.

Qualitative behaviour. The $k = 0$ (fundamental) mode $\sin(\pi x)$ has the slowest decay rate $\pi^2 + \alpha \approx 9.87 + \alpha$; all higher modes decay at least $9\times$ faster. By $t \approx 0.1$ the solution is already dominated by $\sin(\pi x)$ regardless of the initial shape. The discontinuity in the first derivative of `f2` at $x = 1/2$ causes a Gibbs-like transient in coarse simulations, which is a useful test of the schemes' robustness.

---

2. Grid and Discretization

2.1 Grid

We introduce a uniform spatial step $\Delta x = 1/(N_x + 1)$ and time step $\Delta t = T/N_t$:

$$x_i = i\,\Delta x,\ i = 0, \ldots, N_x + 1; \quad t^n = n\,\Delta t,\ n = 0, \ldots, N_t. \tag{14}$$

The N_x interior nodes $x_1, \ldots, x_{N_x}$ carry the unknowns; boundary values $u(x_0, t) = u(x_{N_x+1}, t) = 0$ are enforced exactly and are **not stored**.

The *mesh Fourier number* (sometimes called the diffusion number or CFL for diffusion) is

$$r = \frac{\Delta t}{\Delta x^2}. \tag{15}$$

This single dimensionless parameter controls both accuracy and stability of the explicit scheme, and influences the conditioning of the linear systems in the implicit schemes.

2.2 Second-Order Central Difference for u_{xx}

Applying a standard Taylor expansion:

$$u_{xx}(x_i, t^n) \approx \frac{u_{i-1}^n - 2u_i^n + u_{i+1}^n}{\Delta x^2} + O(\Delta x^2). \tag{16}$$

All three schemes use this same spatial stencil; they differ only in which time level is used to evaluate the right-hand side.

---

3. Numerical Schemes

3.1 Explicit Scheme (FTCS)

Replace u_t by a forward difference and u_{xx} by a centred difference at the current time level n:

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \frac{u_{i-1}^n - 2u_i^n + u_{i+1}^n}{\Delta x^2} - \alpha\, u_i^n. \tag{17}$$

Solving for u_i^{n+1}:

$$u_i^{n+1} = u_i^n + r(u_{i-1}^n - 2u_i^n + u_{i+1}^n) - \alpha\,\Delta t\, u_i^n. \tag{18}$$

This is a **pure update formula** — no linear system needs to be solved. The computational cost per time step is $O(N_x)$ operations.

Matrix form. Defining $\mathbf{U}^n = (u_1^n, \ldots, u_{N_x}^n)^\top$, the update reads

$$\mathbf{U}^{n+1} = A_{\text{FTCS}}\,\mathbf{U}^n, \tag{19}$$

where A_{FTCS} is the $N_x \times N_x$ tridiagonal matrix

$$A_{\text{FTCS}} = (1 - 2r - \alpha\Delta t)\,I + r\,T, \quad T = \text{tridiag}(1,\ 0,\ 1). \tag{20}$$

Explicitly:

$$A_{\text{FTCS}} = \begin{bmatrix} 1-2r-\alpha\Delta t & r & & & \\ r & 1-2r-\alpha\Delta t & r & & \\ & \ddots & \ddots & \ddots & \\ & & r & 1-2r-\alpha\Delta t \end{bmatrix}. \tag{21}$$

The homogeneous Dirichlet conditions $u_0 = u_{N_x+1} = 0$ are incorporated exactly: the boundary terms $r \cdot u_0$ and $r \cdot u_{N_x+1}$ vanish and are absent from the first and last rows.

Local truncation error:

$$\tau_{\text{FTCS}} = O(\Delta t) + O(\Delta x^2). \tag{22}$$

The scheme is *first-order in time* and *second-order in space*.

3.2 Implicit Scheme (BTCS)

Replace u_{xx} at time level $n+1$ (future) rather than n:

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \frac{u_{i-1}^{n+1} - 2u_i^{n+1} + u_{i+1}^{n+1}}{\Delta x^2} - \alpha\, u_i^{n+1}. \tag{23}$$

Rearranging yields a tridiagonal system for the N_x unknowns $\mathbf{u}^{n+1}$:

$$-r\,u_{i-1}^{n+1} + (1 + 2r + \alpha\Delta t)\,u_i^{n+1} - r\,u_{i+1}^{n+1} = u_i^n. \tag{24}$$

In matrix form $A\mathbf{u}^{n+1} = \mathbf{u}^n$ where

$$A = \text{tridiag}(-r,\ 1 + 2r + \alpha\Delta t,\ -r). \tag{25}$$

The matrix A is strictly diagonally dominant (diagonal entry $1 + 2r + \alpha\Delta t > 2r$), hence invertible and well-conditioned for all $r > 0$.

The **Thomas algorithm** (a specialised Gaussian elimination for tridiagonal systems) solves (25) in $O(N_x)$ operations without pivoting, exploiting the sparsity structure.

Local truncation error: $O(\Delta t) + O(\Delta x^2)$ — same order as FTCS.

3.3 Crank-Nicolson Scheme

CN averages the spatial operator at levels n and $n+1$:

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \frac{1}{2}\left[\frac{\delta_x^2 u_i^n}{\Delta x^2} + \frac{\delta_x^2 u_i^{n+1}}{\Delta x^2}\right] - \frac{\alpha}{2}\left(u_i^n + u_i^{n+1}\right), \tag{26}$$

where $\delta_x^2 u_i = u_{i-1} - 2u_i + u_{i+1}$. Rearranging:

Left side (unknowns at $n+1$):

$$-\tfrac{r}{2}\,u_{i-1}^{n+1} + \left(1 + r + \tfrac{\alpha\Delta t}{2}\right)u_i^{n+1} - \tfrac{r}{2}\,u_{i+1}^{n+1}$$

Right side (known at n):

$$\tfrac{r}{2}\,u_{i-1}^{n} + \left(1 - r - \tfrac{\alpha\Delta t}{2}\right)u_i^{n} + \tfrac{r}{2}\,u_{i+1}^{n}$$

In matrix form $A_{\text{CN}}\mathbf{u}^{n+1} = B_{\text{CN}}\mathbf{u}^n$ where both A_{CN} and B_{CN} are tridiagonal. The right-hand side is evaluated explicitly, then the same Thomas algorithm solves the left-hand tridiagonal system.

Local truncation error: $O(\Delta t^2) + O(\Delta x^2)$.

CN is **second-order in both time and space** — one order higher in time than FTCS/BTCS, at the cost of evaluating both sides of the system per step.

---

4. Stability Analysis (Von Neumann)

A necessary condition for stability of a linear scheme on a periodic domain is that the amplification factor G satisfies $|G| \le 1$ for all wavenumbers. We write $u_i^n = G^n e^{i\xi x_i}$ with wavenumber ξ and substitute into each scheme.

4.1 FTCS

Substituting into (18):

$$G = 1 - 4r\sin^2\left(\frac{\xi\Delta x}{2}\right) - \alpha\Delta t. \tag{27}$$

Let $\theta \in [0, \pi]$ with $\theta = \xi\Delta x$. The real amplification factor is

$$G(\theta) = 1 - 4r\sin^2(\theta/2) - \alpha\Delta t. \tag{28}$$

For stability $-1 \le G \le 1$:

- Upper bound $G \le 1$: automatic since $4r\sin^2 \ge 0$ and $\alpha\Delta t \ge 0$.
- Lower bound $G \ge -1$: worst case at $\sin^2(\theta/2) = 1$:

$$1 - 4r - \alpha\Delta t \ge -1 \Rightarrow r \le \frac{1}{2} - \frac{\alpha\Delta t}{4}. \tag{29}$$

For small Δt and $\alpha \ge 0$ this simplifies to the standard **stability condition**

$$r \le \frac{1}{2}. \tag{30}$$

The reaction term $\alpha\Delta t \ge 0$ actually makes the condition *slightly easier* to satisfy — the decay term reduces $|G|$ for all modes. Violating (30) causes the high-frequency mode $\theta = \pi$ to be amplified without bound, producing the overflow behaviour observed numerically (see §6.1).

4.2 BTCS

Substituting into (24):

$$G = \frac{1}{1 + 4r\sin^2(\theta/2) + \alpha\Delta t}. \tag{31}$$

Since the denominator is always ≥ 1, we have $0 < G \le 1$ for all $r > 0$, $\alpha \ge 0$. BTCS is **unconditionally stable**.

4.3 Crank-Nicolson

$$G = \frac{1 - 2r\sin^2(\theta/2) - \alpha\Delta t/2}{1 + 2r\sin^2(\theta/2) + \alpha\Delta t/2}. \tag{32}$$

Both numerator and denominator are real. Write $\mu = 2r\sin^2(\theta/2) + \alpha\Delta t/2 \geq 0$. Then $G = (1-\mu)/(1+\mu)$, which satisfies $-1 < G \leq 1$ for all $\mu \geq 0$. CN is **unconditionally stable**.

However, for large Δt the factor μ can exceed 1, giving $G < 0$. A negative amplification factor reverses sign at each step, producing temporal oscillations on a $2\Delta t$ period. These oscillations are *stable* (they don't grow) but can look like numerical noise if the time step is much too large. In practice, CN should be used with moderate r to suppress these oscillations.

4.4 Summary

Scheme	Order (Δt)	Order (Δx)	Stability
FTCS	1	2	Conditional: $r \leq 1/2$
BTCS	1	2	Unconditional
CN	2	2	Unconditional

---

5. Implementation

5.1 File Structure

```
main.py
schemes/
    explicit.py       # FTCS step function
    implicit.py       # Thomas algorithm + BTCS solver
    crank_nicolson.py # CN RHS builder + solver
utils/
    grid.py           # dx, dt, r computation
    initial_conditions.py  # f1, f2, exact solution
    solver.py         # dispatch layer
    visualization.py  # snapshot plots, validation figures
```

5.2 Thomas Algorithm

For a tridiagonal system with lower diagonal a, main diagonal d, upper diagonal c, and right-hand side b, the Thomas algorithm proceeds as follows.

Forward sweep (eliminate sub-diagonal):

$$c'_0 = c_0/d_0, \quad b'_0 = b_0/d_0, \tag{33}$$

$$c'_i = \frac{c_i}{d_i - a_i c'_{i-1}}, \quad b'_i = \frac{b_i - a_i b'_{i-1}}{d_i - a_i c'_{i-1}}, \quad i = 1, \ldots, N_x - 1. \tag{34}$$

Back substitution:

$$x_{N_x-1} = b'_{N_x-1}, \quad x_i = b'_i - c'_i x_{i+1}, \quad i = N_x - 2, \ldots, 0. \tag{35}$$

The algorithm requires $2 \times (N_x - 1)$ divisions and $4 \times (N_x - 1)$ multiplications/additions — strictly $O(N_x)$ — and stores only three extra arrays of length N_x. No pivoting is needed because the matrix A (BTCS) and A_{CN} are both strictly diagonally dominant throughout the computation.

For BTCS, $d_i = 1 + 2r + \alpha\Delta t$ and $a_i = c_i = -r$ are constant across all rows, so the modified coefficients c'_i need only be computed once (they are the same for every time step), saving $O(N_x)$ work per step if pre-computed. The current implementation recomputes them each step for clarity; this could be optimised for large N_t.

5.3 Boundary Condition Enforcement

Interior unknowns $u_1, \ldots, u_{N_x}$ are stored; boundary values $u_0 = u_{N_x+1} = 0$ are applied implicitly by setting $a_1 = 0$ and $c_{N_x} = 0$ (i.e., the tridiagonal system already incorporates them in its first and last equations). The explicit update uses guard conditions `left=0` for $i = 0$ and `right=0` for $i = N_x - 1$, identical in effect.

---

6. Numerical Results

All experiments use $\alpha = 1.0$ unless noted.

6.1 Stability Demonstration

With $N_x = 100$ and $N_t = 500$, the mesh ratio is $r = 10.20 \gg 0.5$. FTCS produces IEEE overflow (NaN) within a few steps, while BTCS and CN remain fully stable and converge to the exact solution.

To obtain a stable FTCS result we require $N_t \geq \lceil T N_x^2 / (2 \cdot 1/2) \rceil$. For $N_x = 100$, $T = 0.5$: $N_t \geq 2601$ — more than five times the number of steps needed by BTCS/CN for equal spatial resolution. This reflects the well-known cost of explicit schemes for diffusion problems: the time step must scale as $\Delta t \sim \Delta x^2$, so halving the spatial step requires four times as many time steps.

6.2 Spatial Convergence (All Schemes)

With $N_t = 10{,}000$ (time error negligible), varying N_x shows $O(\Delta x^2)$ convergence for all three schemes, as expected from the second-order central difference:

N_x	Δx	FTCS L2	BTCS L2	CN L2	Observed rate
5	0.16667	5.1e-3	5.1e-3	5.1e-3	—
10	0.09091	1.4e-3	1.4e-3	1.4e-3	≈1.90
20	0.04762	3.6e-4	3.8e-4	3.7e-4	≈1.93
40	0.02439	8.6e-5	1.1e-4	9.6e-5	≈1.99
80	0.01235	1.4e-5	3.5e-5	2.4e-5	≈2.00

At coarser grids all schemes yield nearly identical errors, confirming that for N_t large enough the spatial discretisation dominates and the schemes are indistinguishable in accuracy.

6.3 Temporal Convergence

With $N_x = 500$ (spatial error $\lesssim 10^{-6}$, negligible), varying N_t:

Crank-Nicolson (expected O(dt²)):

N_t	Δt	L2 error	Observed rate
5	0.0200	2.69e-3	—
10	0.0100	6.80e-4	1.98
20	0.0050	1.70e-4	2.00
40	0.0025	4.20e-5	2.02
80	0.0013	1.00e-5	2.07

The clean rate of **2.0** confirms second-order accuracy in time for CN.

BTCS (expected O(dt)):

N_t	Δt	L2 error	Observed rate
5	0.0200	2.41e-2	—
10	0.0100	1.13e-2	1.10
20	0.0050	5.38e-3	1.07
40	0.0025	2.61e-3	1.04
80	0.0013	1.29e-3	1.02
160	0.0006	6.38e-4	1.01

The rate converges cleanly to **1.0**.

The temporal error of BTCS at $N_t = 5$ is about 35× larger than CN at the same N_t. To match CN's accuracy at $N_t = 10$ ($L_2 \approx 6.8 \times 10^{-4}$), BTCS needs $N_t \approx 35$, i.e., $3.5\times$ more steps.

6.4 Scheme Comparison ($N_x = 100$, stable $N_t = 5{,}000$, $r = 1.02$)

Scheme	L2 error	L^∞ error	Comment
FTCS	NaN	NaN	Unstable ($r = 1.02 > 0.5$)
BTCS	5.5e-11	7.8e-11	First-order in t
CN	7.3e-12	1.0e-11	Second-order in t

Both implicit schemes reach near-machine-precision due to the very small Δt; CN is consistently ~7× more accurate than BTCS, consistent with the ratio $(\Delta t)^1/(\Delta t)^2 = 1/\Delta t$ advantage of second-order accuracy.

6.5 Effect of α

As α increases, the solution decays faster but the spatial structure is unchanged (for `f1`). The numerical schemes remain accurate across the tested range:

α	$\max\lvert u_{\text{num}}\rvert$ at $T = 0.5$	CN L2 error
0.0	2.7e-9	7.2e-12
0.5	2.1e-9	5.5e-12
1.0	1.6e-9	4.1e-12
5.0	2.2e-10	4.2e-13
10.0	1.8e-11	1.7e-14

Larger α damps the solution to near-zero faster, so absolute errors also decrease — the problem becomes easier numerically as α grows. The relative accuracy remains excellent throughout.

---

7. Discussion

7.1 Stability vs. Computational Cost

The CFL-like constraint $r \le 1/2$ for FTCS forces $\Delta t \le \Delta x^2/2$. For a spatial resolution of $N_x = 100$ ($\Delta x \approx 0.01$), a stable FTCS simulation of $T = 0.5$ requires at least $N_t \approx 2601$ steps. BTCS and CN need only as many steps as dictated by *accuracy*, not stability — and for smooth solutions, accuracy is achieved with far fewer steps.

This disparity grows as N_x increases: doubling the spatial resolution quadruples the required number of FTCS steps (due to the $\Delta t \sim \Delta x^2$ constraint) while BTCS/CN may need only twice as many steps to maintain $O(\Delta t)$ accuracy. For three-dimensional problems, this advantage is even more pronounced.

7.2 Accuracy Trade-Off Between BTCS and CN

Both BTCS and CN solve one tridiagonal system per step. The extra computation in CN is building the right-hand side $B_{\text{CN}}\mathbf{u}^n$, which requires one additional $O(N_x)$ pass — roughly doubling the arithmetic per step. Against this modest overhead, CN gains a full order in time accuracy.

The break-even point: to achieve a target temporal error ε in time T, BTCS requires $N_t \sim T\varepsilon^{-1}$ steps while CN needs $N_t \sim \sqrt{T\varepsilon^{-1}}$ steps. For $\varepsilon = 10^{-4}$, $T = 0.5$: BTCS needs $N_t \sim 5000$, CN needs $N_t \sim 71$. The savings factor is ~ 70, far outweighing the factor-2 overhead. **CN is the preferred scheme for smooth solutions.**

7.3 Numerical Diffusion

Numerical diffusion (also called numerical dissipation) refers to the extra amplitude damping introduced by the discretisation beyond that predicted by the exact PDE. It can be quantified by comparing the numerical amplification factor G with the exact one $G_{\text{exact}} = e^{-(\lambda+\alpha)\Delta t}$, where $\lambda = 4r\sin^2(\theta/2)/\Delta t$ is the spatial eigenvalue contribution.

BTCS. The backward-Euler time discretisation replaces the exact factor $e^{-(\lambda+\alpha)\Delta t}$ by $1/(1+(\lambda+\alpha)\Delta t)$. For small $\xi = (\lambda+\alpha)\Delta t$ a Taylor expansion gives

$$\frac{1}{1+\xi} = e^{-\xi}\cdot e^{-\xi^2/2+O(\xi^3)} \approx e^{-\xi}\,e^{-\xi^2/2}, \tag{36}$$

meaning BTCS *over-damps* modes by an extra factor $e^{-\xi^2/2}$: each Fourier mode decays faster than the exact solution. This is **first-order numerical diffusion** — the error in the amplification factor is $O(\Delta t^2)$ per step, accumulating to $O(\Delta t)$ over a fixed time T.

FTCS. The forward-Euler factor $1-\xi = e^{-\xi}e^{\xi^2/2+O(\xi^3)}$ is the mirror image: modes are *under-damped* when ξ is small and the scheme is still stable. Paradoxically, FTCS introduces *less* numerical diffusion than BTCS near the stability limit but becomes unstable when $r > 1/2$.

Crank-Nicolson. The bilinear (Padé) approximation gives

$$\frac{1-\xi/2}{1+\xi/2} = e^{-\xi}\cdot e^{\xi^3/12+O(\xi^5)}, \tag{37}$$

so the extra damping is only $O(\Delta t^3)$ per step, accumulating to $O(\Delta t^2)$ — consistent with second-order accuracy in time. CN introduces negligible numerical diffusion for moderate Δt.

Practical consequence. For the tent function `f2`, whose high-frequency Fourier modes carry the kink at $x = 1/2$, the excess damping in BTCS causes the kink to smooth out faster than the exact solution. CN preserves the amplitude of each mode much more faithfully. This is directly visible in the comparison plots: BTCS solutions appear slightly smoother than CN solutions at the same N_t.

7.4 When to Use Each Scheme

- **FTCS**: Instructive for learning; useful when $r \ll 1/2$ (very coarse time steps are needed anyway for accuracy, and no linear solve is needed). Can be competitive for nonlinear problems where implicit solves are expensive.
- **BTCS**: Robust and simple; preferred when stability is paramount and temporal accuracy requirements are modest, or as an inner solver in splitting methods.
- **CN**: Best overall accuracy-efficiency for smooth, well-resolved problems. Standard choice in production heat-equation solvers. Should be used with moderate r to avoid temporal oscillations (which are stable but aesthetically undesirable).

7.5 The Reaction Term

A noteworthy observation: the $-\alpha u$ term does not alter the spatial modes, it only adjusts their temporal decay rate from λ_k to $\lambda_k + \alpha$. For numerical purposes, $\alpha > 0$ *improves* stability and conditioning:

- In FTCS, the amplification factor $G = 1 - 4r\sin^2(\theta/2) - \alpha\Delta t$ is strictly less than 1 for $\alpha > 0$, relaxing the lower-bound constraint.
- In BTCS/CN, larger α increases the diagonal dominance of A, improving the conditioning of the tridiagonal system.

This behaviour generalises: for parabolic PDEs with positive reaction, implicit schemes tend to become *more* stable and accurate as the reaction coefficient grows, which is the opposite of the difficulty introduced by stiff reaction terms with $\alpha < 0$ (growth) or in systems of PDEs.

7.6 Behaviour with f2 (Tent Function)

The tent function has a discontinuous first derivative at $x = 1/2$, whose Fourier series converges only as $O(k^{-2})$ (rather than the exponential convergence for smooth data). This means:

1. Coarse grids will misrepresent the initial kink.
2. The early-time solution shows the kink smoothing by diffusion — a physically meaningful phenomenon (the PDE regularises the data immediately for $t > 0$).
3. All three schemes handle this correctly, but CN at large Δt can produce visible oscillations near the kink due to the $G < 0$ effect for high wavenumbers (modes with $\mu > 1$).

This makes `f2` a more discriminating test than `f1` for assessing scheme behaviour near non-smooth data.

---

8. Conclusion

We implemented and validated three finite-difference schemes for the reaction-diffusion equation $u_t = u_{xx} - \alpha u$ on the unit interval. The key findings are:

1. **Stability is the dominant constraint for FTCS**: the condition $r \leq 1/2$ limits practical use to coarse spatial grids or very short time intervals, unless accuracy requires small Δt independently.
2. **BTCS and CN are unconditionally stable and practically interchangeable in stability terms**, but CN achieves second-order accuracy in time at the cost of one extra $O(N_x)$ pass per step — a highly favourable trade-off.
3. **Numerical experiments confirm theoretical orders**: O(dt²+dx²) for CN and O(dt+dx²) for BTCS and FTCS, both with spatial convergence rate ≈ 2.
4. **The Thomas algorithm** solves the tridiagonal systems in $O(N_x)$ time with no pivoting required, making the per-step cost of BTCS and CN comparable to FTCS.
5. **The reaction term $\alpha > 0$ is benign**: it improves diagonal dominance of the implicit systems and accelerates amplitude decay, making all schemes slightly more accurate as α grows.

For practical application, **Crank-Nicolson is recommended** as the default scheme: it combines unconditional stability, second-order accuracy, and efficient implementation via the

Thomas algorithm, making it the best balance of robustness and efficiency among the three methods considered.

Appendix: Usage

```
# Single scheme run
python3 main.py --scheme crank-nicolson --Nx 100 --Nt 500 --alpha 1.0 --T 0.5 --initial
f1

# Validate against exact solution
python3 main.py --scheme crank-nicolson --Nx 100 --Nt 500 --alpha 1.0 --T 0.5 --initial
f1 --validate

# Compare all three schemes
python3 main.py --compare --Nx 100 --Nt 500 --alpha 1.0 --T 0.5 --initial f1

# Stable explicit (r < 0.5 requires large Nt for fine Nx)
python3 main.py --compare --Nx 50 --Nt 3000 --alpha 1.0 --T 0.5 --initial f2
```